

AhamX Bodhi

Where I Becomes Infinite

Feed-Forward Networks in Transformers

Module 3.2: Transformer Architecture Deep-Dive
Prof. Sashikumaar Ganesan | IISc Bengaluru



Session Overview

What You Will Learn

- The role of the FFN in a Transformer block
- The two-layer MLP structure and activations
- Why FFN width is $4 \times d_{\text{model}}$
- GELU, SwiGLU, and modern activation choices
- FFN as a key-value memory store

Concept at a Glance

- **Duration:** 8 minutes
- **Bloom's Level:** B3 – Apply
- **Difficulty:** L2 – Intermediate
- **Module:** 3.2 Transformer Deep-Dive
- **Prerequisite:** Layer Normalisation



Learning Objectives

By the End of This Session You Will Be Able To

1. **Describe** the structure of the position-wise FFN and its role in a Transformer block
2. **Explain** why the FFN uses an inner dimension of $4 \times d_{\text{model}}$
3. **Compare** ReLU, GELU, and SwiGLU activations and state which is used in modern LLMs
4. **Interpret** the FFN as a key-value associative memory
5. **Identify** the FFN's contribution to total parameter count in GPT and Llama models



Prerequisites and Connections

You Should Already Know

- Multi-layer perceptron (MLP) structure
- Non-linear activation functions (ReLU)
- Residual connections and Layer Normalisation
- Forward pass of a Transformer block

This Concept Unlocks

- Full Transformer block assembly (coding)
- Mixture-of-Experts (MoE) FFN variants
- LoRA applied to FFN layers
- Knowledge editing through FFN weights





The FFN in a Transformer Block

Position of the FFN in the Block

Every Transformer block (encoder or decoder) contains two sub-layers:

1. **(Masked) Multi-Head Attention:** Mixes information across sequence positions — each token can “look at” other tokens
2. **Feed-Forward Network:** Applied **independently** to each token position — no information mixing across positions at this step

Each sub-layer is wrapped with a residual connection and Layer/RMS Normalisation. The FFN acts as a **per-position transformation**: the same MLP parameters are applied to every token's representation independently.

Attention mixes *where* to look; the FFN decides *what to do* with what was found.



The FFN Formula

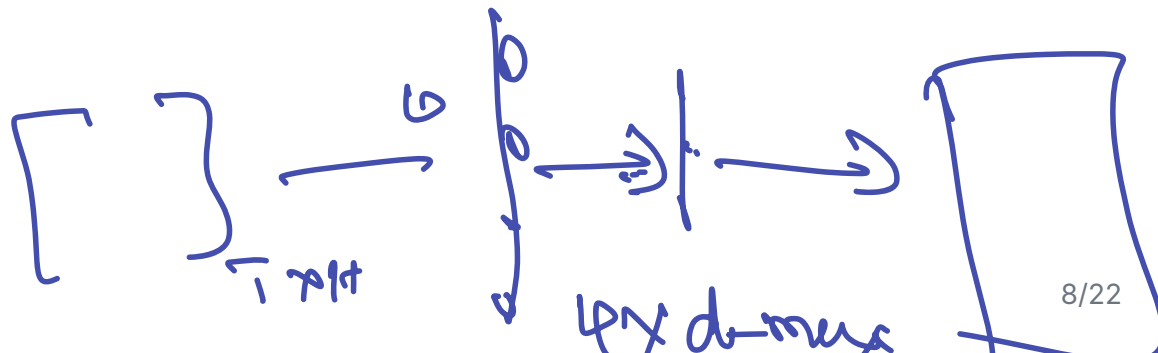
A Two-Layer MLP With One Non-linearity

The position-wise FFN transforms each token representation $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$:

$$\text{FFN}(\mathbf{x}) = W_2 \cdot \sigma(W_1 \mathbf{x} + b_1) + b_2$$

where $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $d_{\text{ff}} = 4 \times d_{\text{model}}$, and σ is a non-linear activation.

The output shape is the same as the input: $\mathbb{R}^{d_{\text{model}}}$ per token.





Why $d_{\text{ff}} = 4 \times d_{\text{model}}$?

The 4x Expansion is Empirically and Theoretically Motivated

- The bottleneck-then-expand pattern is a well-known principle: project to a higher-dimensional space where features are more separable, then project back
- A larger inner dimension means more “neurons” that can specialise in representing different factual patterns and linguistic phenomena
- The original paper’s ablations showed that reducing to $2\times$ or $1\times$ substantially hurt translation quality; $4\times$ was the empirical sweet spot
- In modern LLMs: for $d_{\text{model}} = 4096$ (Llama 3-8B), $d_{\text{ff}} = 14,336$ (approx. $3.5\times$ with SwiGLU adjustment)

The FFN contains approximately $2/3$ of total model parameters at a fixed depth ratio.



Activation Functions in the FFN

Activation	Formula / Description	Used In		Advantage
ReLU	$\max(0, x)$	Original former, GPT	Trans- early	Simple, fast
GELU	$x \cdot \Phi(x)$ (Gaussian CDF)	BERT, GPT-3	GPT-2,	Smooth, bet- ter than ReLU
SiLU / Swish	$x \cdot \sigma(x)$	Some Transformers	vision	Self-gated
SwiGLU	$\text{SiLU}(W_1 x) \odot (W_3 x)$	Llama PaLM, Mistral	2/3,	Best empir- ical perfor- mance
GeGLU	$\text{GELU}(W_1 x) \odot (W_2 x)$	T5 v1.1, GPT variants	some	Strong, widely tested



SwiGLU: The Modern Standard Activation

Gated Linear Unit with Swish (Noam Shazeer, 2020)

- Replaces the single-activation FFN with a **gated** variant:
$$\text{FFN}_{\text{SwiGLU}}(\mathbf{x}) = (W_1 \mathbf{x} \odot \text{SiLU}(W_3 \mathbf{x})) \cdot W_2$$
- Three weight matrices instead of two (W_1, W_2, W_3)
- The gate $\text{SiLU}(W_3 \mathbf{x})$ controls how much of each feature passes through
- To maintain the same parameter count as a $4\times$ FFN, the inner dimension is set to $\frac{2}{3} \times 4 \times d_{\text{model}}$ (three matrices, same total params)

SwiGLU consistently improves perplexity over GELU at the same parameter budget. This is why Llama 3-8B uses $d_{\text{ff}} = 14,336 \approx 3.5 \times 4096$.



FFN as a Key-Value Associative Memory

Geva et al. (2021): "Transformer Feed-Forward Layers Are Key-Value Memories"

- Each row of W_1 acts as a **key**: it fires (activates) when the input matches a specific pattern (factual association, syntactic template)
- The corresponding row of W_2 acts as a **value**: the output to be added when that key fires
- The non-linearity acts as a **threshold**: only neurons that strongly match the input contribute to the output
- Lower layers: syntactic and positional patterns; Upper layers: factual associations ("Paris" → "France", "Einstein" → "physics")

This memory interpretation explains why editing individual FFN weights can change specific facts stored in the model without retraining.





GenAI Application: Mixture-of-Experts FFN

Scaling Parameters Without Scaling Compute

- **Mixtral 8×7B (Mistral AI):** The FFN in every Transformer block is replaced by 8 independent expert FFNs; a routing network selects the top-2 experts for each token — only 2/8 experts compute per token
- **GPT-4 (rumoured MoE):** 8 expert FFNs per block, routing top-2 per token; 1.8T total parameters but only ~220B active per forward pass
- **DeepSeek-V3 (671B MoE):** 256 experts per FFN block, top-8 active; achieves GPT-4 level performance at fraction of active parameter cost
- **Advantage:** Total capacity scales with number of experts; inference cost scales only with active experts (not total)

MoE transforms the FFN from a dense MLP to a sparse, conditional computation graph.



GenAI Application: Model Editing and Knowledge Update

Surgically Updating Factual Knowledge in LLMs

- **ROME (Rank-One Model Editing):** Identifies which FFN layer stores a specific fact (e.g. "Eiffel Tower is in Paris") and edits that single weight matrix to change the fact — without retraining
- **MEMIT:** Extends ROME to batch-edit thousands of facts simultaneously by distributing edits across multiple FFN layers
- **Use case:** Updating LLMs for new events (election results, product launches) without full fine-tuning — critical for keeping enterprise models current
- **Limitation:** Edits can cause ripple effects; interpretability of FFN key-value structure is still an active research area



GenAI Application: Mechanistic Interpretability

Understanding What FFN Neurons Learn

- **Anthropic feature interpretability:** Identified 34 million monosemantic features in Claude 3 Sonnet's MLP neurons using sparse autoencoders (SAEs); features correspond to named entities, syntax patterns, and concepts
- **Neuron activation steering:** Activating specific FFN neurons can predictably alter model behaviour (e.g. trigger sentiment shifts, factual biases)
- **Superposition hypothesis:** FFN layers pack more features than neuron count by representing multiple features per neuron in superposition; SAEs are the key tool to decompose this
- **Safety implications:** Understanding FFN feature structure is a foundation for detecting and removing harmful capabilities from LLMs



GenAI Application: FFN Pruning and Efficient Inference

Reducing FFN Size for Production Deployment

- **Structured pruning:** Many FFN neurons are near-zero for most inputs; pruning the lowest-magnitude rows of W_1 and W_2 reduces latency with minimal accuracy loss (SparseGPT, Wanda)
- **Activation sparsity:** ReLU creates $\sim 90\%$ sparse activations; exploiting this sparsity in custom CUDA kernels dramatically reduces FFN FLOPs
- **Deja Vu (Liu et al. 2023):** Predicts which FFN neurons will activate *before* running the FFN; skips the rest — 2x speedup on LLaMA
- **FFN quantisation:** W_1 and W_2 quantised to 4-bit (AWQ, GPTQ); the FFN accounts for most quantisation savings at low bit widths



FFN Parameter Count in Modern LLMs

FFN Dominates Total Parameter Budget

For a Transformer with L layers, d_{model} , $d_{\text{ff}} = 4d_{\text{model}}$:

- **Per-layer FFN parameters:** $2 \times d_{\text{model}} \times d_{\text{ff}} = 8d_{\text{model}}^2$ (two matrices)
- **Per-layer Attention parameters:** $4 \times d_{\text{model}}^2$ (four matrices: W^Q, W^K, W^V, W^O)
- **Ratio:** FFN holds $2\times$ as many parameters as attention per layer
- **Llama 3-8B:** $d_{\text{model}} = 4096$, $L = 32$; FFN accounts for $\approx 65\%$ of total parameters

Implication for LoRA

LoRA targets attention weight matrices by default but recent work (LoRA+, DoRA) shows applying LoRA to FFN layers significantly improves fine-tuning performance.



Common Misconceptions

Clearing Up Confusion About FFNs in Transformers

- **“The FFN is unimportant compared to attention”:** The FFN holds $\sim 2/3$ of model parameters and stores factual knowledge. Ablation studies show removing FFN layers causes severe performance degradation — more so than removing attention heads.
- **“The FFN operates on the whole sequence at once”:** The FFN is *position-wise* — the same function applied independently to each token. There is no interaction between positions in the FFN step.
- **“Larger inner dimension always helps”:** Beyond a certain ratio, returns diminish and computational cost increases. MoE is a more efficient way to scale capacity than simply increasing d_{ff} .



Feed-Forward Networks in Transformers — Key Takeaways

1. The FFN is a two-layer position-wise MLP applied independently to each token; it does not mix information across positions
2. The inner dimension $d_{\text{ff}} = 4 \times d_{\text{model}}$ creates an expansion space where richer feature representations form
3. GELU was standard (BERT, GPT-3); SwiGLU is now preferred (Llama 3, Mistral) with a three-matrix gated formulation at $\sim 3.5\times$ inner dimension
4. The FFN can be interpreted as a key-value associative memory that stores factual associations and linguistic patterns in its weight matrices
5. Mixture-of-Experts replaces the dense FFN with sparse conditional experts, scaling model capacity without proportionally scaling compute



What This Concept Unlocks

Next in Module 3.2

- Full Transformer Implementation (coding)
- Complete Transformer block assembly

Later in the Course

- Mixture-of-Experts Architecture
- LoRA Applied to FFN Layers
- Mechanistic Interpretability
- Model Editing (ROME, MEMIT)

Thank You

Feed-Forward Networks in Transformers | Module 3.2 | AhamX
Bodhi